I am going to build a real-world [ANDROID APP / IOS APP / WEB APP / SAAS / FULL-STACK PRODUCT].

Your job is to act as my:

• Senior Product Manager
• Business Analyst
• UX/UI Strategist
• Software Architect
• Senior Full-Stack Engineer
• Database Architect
• Security Engineer
• Privacy & Compliance Researcher
• QA/Test Engineer
• DevOps/Cloud Architect
• App Store/Play Store Launch Expert
• Technical Documentation Writer

I want to take this product from IDEA → MVP → PRODUCTION → LAUNCH → SCALE.

Do NOT give me a shallow generic answer.

Perform extensive research before making recommendations. When current information matters, use reliable and up-to-date sources. Do not invent laws, APIs, pricing, platform requirements, limits, or technical capabilities.

==========
1. UNDERSTAND THE PRODUCT
==========

First analyze my idea:

[PASTE MY PRODUCT IDEA HERE]

Explain:

• What problem it solves
• Who the target users are
• Primary use cases
• Secondary use cases
• User personas
• User pain points
• Core value proposition
• What makes the product different
• Assumptions being made
• Risks and unknowns
• Potential abuse/misuse

• Whether any part of the idea requires special legal, privacy, security, financial, medical, AI, or regulatory consideration

If something important is unclear, identify the assumption you are making instead of silently inventing requirements.

==========
2. REQUIREMENTS DOCUMENT / PRD
==========

Create a complete Product Requirements Document.

Include:

• Product overview
• Goals
• Non-goals
• Target users
• User stories
• Functional requirements
• Non-functional requirements
• MVP requirements
• Post-MVP features
• Future features
• Feature priorities
• Dependencies
• Constraints
• Success metrics
• KPIs
• Acceptance criteria

For every major feature explain:

1. What it does
2. Why it exists
3. User flow
4. Required inputs
5. Outputs
6. Edge cases
7. Error states
8. Permissions required
9. Data generated/stored
10. Security/privacy implications

Clearly separate:

MUST HAVE
SHOULD HAVE

COULD HAVE
NOT REQUIRED FOR MVP

==========
3. COMPLETE USER FLOWS
==========

Map the entire product journey.

Include:

• First launch
• Onboarding
• Authentication
• Main experience
• Every major feature
• Empty states
• Loading states
• Error states
• Offline states
• Permission denial
• Account deletion
• Logout
• Data deletion
• Payment/subscription flow if applicable
• Cancellation/refund flow if applicable
• Support/contact flow

Represent important flows step-by-step.

==========
4. SCREEN-BY-SCREEN SPECIFICATION
==========

Create a complete screen inventory.

For EVERY screen specify:

• Screen name
• Purpose
• UI components
• Navigation
• Buttons/actions
• Input fields
• Validation
• Loading state
• Empty state
• Error state

• Success state
• Permission requirements
• API calls
• Database interaction
• Analytics events
• Accessibility considerations

Do not forget hidden screens such as:

• Settings
• Privacy
• Terms
• Help
• Contact
• Account deletion
• Error screens
• Offline screens
• Permission screens
• Subscription management
• Legal disclosures

==========
5. UX/UI SYSTEM
==========

Define:

• Design philosophy
• Information architecture
• Navigation structure
• Typography
• Spacing system
• Components
• Buttons
• Forms
• Cards
• Modals
• Toasts
• Dialogs
• Bottom sheets
• Icons
• Animations
• Loading indicators
• Empty states
• Error states
• Dark mode/light mode if applicable
• Responsive behavior
• Accessibility

Make the UI practical to implement, not just visually attractive.

==========
6. TECH STACK RESEARCH
==========

Recommend the technology stack.

Compare reasonable alternatives before choosing.

Cover:

• Frontend
• Backend
• Database
• Authentication
• File storage
• API architecture
• Hosting
• CDN
• Caching
• Search
• Background jobs
• Notifications
• Email
• Analytics
• Crash reporting
• Logging
• Monitoring
• Payments
• AI/LLM APIs if required
• Maps/location if required
• Third-party integrations

For each important technology explain:

• Why it is suitable
• Advantages
• Disadvantages
• Cost
• Free-tier limitations
• Scaling concerns
• Vendor lock-in
• Security considerations
• Alternative options

Prefer simple, reliable, inexpensive technology for the MVP.

==========
7. SYSTEM ARCHITECTURE
==========

Design the complete architecture.

Include:

• Client
• Backend
• Database
• Authentication
• APIs
• External services
• Storage
• Caching
• Background workers
• Queues
• Monitoring
• Deployment

Explain how data flows through the system.

Provide architecture diagrams using Mermaid where useful.

Example:

User
↓
Frontend
↓
API
↓
Authentication
↓
Business Logic
↓
Database / External Services

==========
8. DATABASE DESIGN
==========

Design the database from scratch.

Include:

• Tables/collections
• Fields
• Data types
• Primary keys
• Foreign keys
• Relationships
• Indexes
• Constraints
• Validation
• Timestamps
• Soft deletion if appropriate

Provide an ER diagram where useful.

Also explain:

• What data is actually necessary
• What data should NOT be stored
• Data retention
• Data deletion
• Backup strategy
• Migration strategy

Follow the principle:

COLLECT THE MINIMUM DATA NECESSARY.

==========
9. API DESIGN
==========

Design the API layer.

For each endpoint include:

• HTTP method
• Endpoint
• Authentication requirement
• Request
• Response
• Validation
• Error responses
• Authorization
• Rate limiting
• Example request
• Example response

Also explain:

• API versioning
• Pagination
• Idempotency
• Rate limiting
• Input validation
• Error handling

==========
10. SECURITY AUDIT
==========

Treat security as a first-class requirement.

Analyze:

• Authentication
• Authorization
• Password security
• Session management
• JWT/token security
• API security
• Database security
• Encryption
• HTTPS
• Secrets management
• Input validation
• SQL/NoSQL injection
• XSS
• CSRF
• SSRF
• IDOR
• Brute-force attacks
• Rate limiting
• Abuse prevention
• File upload security
• Dependency vulnerabilities
• Admin access
• Logging
• Backup security
• Account takeover
• Data leakage

Create a basic threat model.

For every important risk provide:

RISK → IMPACT → MITIGATION

==========
11. PRIVACY & DATA PROTECTION
==========

Determine exactly what personal/user data the product collects.

Create a data inventory:

DATA → PURPOSE → WHERE STORED → WHO CAN ACCESS → RETENTION → DELETION

Check which privacy/data-protection requirements may apply based on:

- User geography
- Business geography
- Type of data
- Age of users
- Product category
- Data processing activities

Consider relevant frameworks/laws such as applicable:

- India's Digital Personal Data Protection framework
- GDPR/UK GDPR where applicable
- CCPA/CPRA where applicable
- COPPA where applicable
- Other jurisdiction-specific requirements

Do NOT assume that a law applies just because it exists.

Explain applicability and uncertainty.

Also determine requirements for:

- Consent
- Privacy notices
- Data access
- Data correction
- Data deletion
- Data portability where applicable
- Data retention
- Children's data
- Sensitive data
- Third-party processors
- International data transfers
- Cookies
- Tracking

• Analytics
• Advertising
• AI processing

==========
12. LEGAL DOCUMENTS
==========

Identify ALL documents the product may need.

Potentially include:

• Privacy Policy
• Terms of Service
• Terms & Conditions
• Cookie Policy
• Refund/Cancellation Policy
• Subscription Terms
• Disclaimer
• Acceptable Use Policy
• Community Guidelines
• Data Processing Agreement
• Consent notices
• Copyright/IP policy
• AI disclosure
• Third-party disclosure
• Medical/financial disclaimer if relevant
• Age restrictions
• User-generated-content policy if relevant

For every document explain:

• Whether it is needed
• Why
• When it should be shown
• Where it should be linked
• What information it needs

IMPORTANT:
Do not present generic legal text as guaranteed legal advice.
Identify areas where a qualified lawyer should review the final wording.

==========
13. COOKIES & TRACKING
==========

Do NOT forget cookies and similar technologies.

Determine:

• Whether cookies are actually required
• Essential cookies
• Analytics cookies
• Advertising cookies
• Session cookies
• Persistent cookies
• Local storage
• Device identifiers
• Tracking technologies
• Consent requirements
• Cookie banner requirements
• Opt-out requirements
• Privacy settings

For mobile apps also consider:

• Advertising IDs
• Device identifiers
• SDK tracking
• Analytics SDKs
• Crash-reporting SDKs
• Push notification tokens

==========
14. THIRD-PARTY SERVICES
==========

Create a complete third-party dependency inventory.

For every service specify:

SERVICE
PURPOSE
DATA SHARED
DATA LOCATION
PRIVACY IMPLICATION
SECURITY IMPLICATION
COST
FREE TIER
LIMITS
ALTERNATIVES

Do not recommend a third-party service without considering what user data it receives.

==========
15. PAYMENTS & MONEY

==========

If payments are involved, analyze:

• Payment gateway
• Payment flow
• One-time payment
• Subscription
• Taxes
• Invoices
• Refunds
• Chargebacks
• Failed payments
• Webhooks
• Payment security
• PCI implications
• Platform fees
• App Store/Play Store billing requirements
• Indian payment considerations where applicable

Clearly distinguish platform billing from external payment gateways.

==========
16. APP STORE / PLAY STORE REQUIREMENTS
==========

If this is a mobile app, research current requirements for:

• Google Play
• Apple App Store

Check:

• Privacy requirements
• Data Safety declarations
• App permissions
• Account deletion
• Payments
• Subscriptions
• User-generated content
• Login requirements
• Advertising
• Age rating
• Store listing
• Screenshots
• App description
• Content policies
• SDK requirements

• Target SDK requirements
• Review requirements

Do not rely on outdated requirements.

==========
17. WEB REQUIREMENTS
==========

If this is a web application/SaaS, analyze:

• Domain
• DNS
• HTTPS
• SSL
• Hosting
• CDN
• robots.txt
• sitemap
• SEO
• Open Graph metadata
• Favicon
• Web manifest if applicable
• Security headers
• CSP
• CORS
• Cookie behavior
• Accessibility
• Responsive design
• Performance
• Core Web Vitals
• Error pages
• Email configuration

==========
18. ANALYTICS
==========

Design analytics without unnecessary surveillance.

Define:

• Events
• Properties
• Conversion funnel
• Retention metrics
• Activation metric
• Revenue metrics

• Crash metrics
• Performance metrics

For every analytics event explain:

EVENT → WHY NEEDED → DATA COLLECTED → PRIVACY IMPLICATION

Do not collect data simply because it is technically possible.

==========
19. TESTING
==========

Create a complete QA strategy.

Include:

• Unit tests
• Integration tests
• API tests
• UI tests
• End-to-end tests
• Security tests
• Accessibility tests
• Performance tests
• Load tests
• Regression tests

Create test cases for:

• Happy paths
• Edge cases
• Invalid inputs
• Network failures
• Empty data
• Duplicate actions
• Authentication failures
• Permission denial
• Payment failures
• Account deletion
• Data deletion

==========
20. PERFORMANCE & SCALABILITY
==========

Design for:

• First 100 users
• First 1,000 users
• 10,000 users
• 100,000 users
• 1,000,000 users

Explain when architectural changes become necessary.

Identify:

• Database bottlenecks
• API bottlenecks
• Storage costs
• Bandwidth costs
• Third-party API limits
• Rate limits
• Caching opportunities
• Queue requirements
• CDN requirements

Do NOT over-engineer the MVP.

==========
21. COST ANALYSIS
==========

Estimate costs at different stages:

MVP
1,000 users
10,000 users
100,000 users
1,000,000 users

Separate:

• Development
• Hosting
• Database
• Storage
• Bandwidth
• Authentication
• Email
• SMS
• AI/API usage
• Analytics
• Monitoring
• Payment fees

• Domain
• App Store fees
• Other third-party services

Clearly identify:

FREE
LOW-COST
USAGE-BASED
REQUIRED PAID

Use current pricing where available and state the date checked.

==========
22. DEVELOPMENT PLAN
==========

Break development into phases:

PHASE 0 — Research
PHASE 1 — Project setup
PHASE 2 — UI
PHASE 3 — Backend
PHASE 4 — Database
PHASE 5 — Authentication
PHASE 6 — Core features
PHASE 7 — Security
PHASE 8 — Testing
PHASE 9 — Deployment
PHASE 10 — Launch
PHASE 11 — Monitoring
PHASE 12 — Scaling

For each phase provide:

• Tasks
• Dependencies
• Deliverables
• Definition of done

==========
23. VIBE-CODING IMPLEMENTATION
==========

I want to actually build this product.

Therefore provide implementation prompts that I can give to an AI coding agent.

Each prompt must:

• Be self-contained
• Specify exactly what to build
• Specify files/components to create or modify
• Follow the existing architecture
• Avoid breaking existing functionality
• Include validation
• Include error handling
• Include security requirements
• Include tests
• Clearly state the expected final result

Build the project incrementally rather than asking the AI to generate the entire application blindly in one step.

==========
24. DEPLOYMENT
==========

Explain:

• Development environment
• Staging environment
• Production environment
• Environment variables
• Secrets
• CI/CD
• Build process
• Database migrations
• Backups
• Monitoring
• Logging
• Crash reporting
• Rollbacks
• Domain setup
• SSL
• Release process

==========
25. LAUNCH CHECKLIST
==========

Create a final launch checklist covering:

PRODUCT
UX
SECURITY

PRIVACY
LEGAL
DATABASE
BACKEND
FRONTEND
PERFORMANCE
ANALYTICS
PAYMENTS
EMAIL
DOMAIN
SEO
APP STORE
PLAY STORE
BACKUPS
MONITORING
SUPPORT
MARKETING

Mark each item:

[ ] NOT STARTED
[ ] IN PROGRESS
[ ] COMPLETE
[ ] NOT APPLICABLE

==========
26. "WHAT DID WE FORGET?" AUDIT
==========

This section is extremely important.

After completing the entire analysis, perform a second independent audit.

Pretend you are a senior engineer reviewing this product immediately before launch.

Ask:

"What could we have forgotten?"

Specifically check for overlooked:

• Features
• Screens
• User flows
• Edge cases
• Security vulnerabilities
• Privacy requirements
• Cookies

• Tracking
• Permissions
• Legal documents
• Data retention
• Data deletion
• Authentication issues
• Payment issues
• Taxes
• App Store requirements
• Play Store requirements
• Accessibility
• SEO
• Analytics
• Monitoring
• Backups
• Disaster recovery
• Rate limits
• Abuse prevention
• Scalability
• Third-party dependencies
• API failures
• Offline behavior
• Account recovery
• Admin functionality
• Support functionality
• Child/age-related requirements
• Internationalization
• Localization
• AI risks
• Cost surprises

Then provide a section:

"TOP 20 THINGS MOST LIKELY TO BE FORGOTTEN"

==========
27. FINAL MASTER DOCUMENT
==========

Finally organize everything into:

1. Executive Summary
2. Product Definition
3. PRD
4. Feature Specification
5. User Flows
6. Screen Specification
7. UX/UI System

8. Technical Architecture
9. Tech Stack
10. Database
11. APIs
12. Security
13. Privacy
14. Cookies & Tracking
15. Legal/Compliance
16. Third-Party Services
17. Payments
18. Analytics
19. Testing
20. Scalability
21. Cost
22. Development Roadmap
23. Vibe-Coding Prompts
24. Deployment
25. Launch Checklist
26. Post-Launch Plan
27. Final "What Did We Forget?" Audit

IMPORTANT RULES:

• Never invent facts.
• Never assume a requirement is current without checking when it may have changed.
• Clearly distinguish facts, recommendations, assumptions, and uncertainties.
• Cite important current/recent claims to reliable primary sources.
• Prefer official documentation, government sources, platform policies, and first-party documentation.
• Do not blindly recommend expensive infrastructure.
• Optimize for simplicity and low cost during MVP.
• Design for scalability without prematurely over-engineering.
• Minimize collection of personal data.
• Treat security and privacy as requirements, not afterthoughts.
• Do not forget cookies, tracking, analytics, permissions, legal documents, or account/data deletion.
• Identify anything that requires professional legal advice rather than pretending generic text is legally sufficient.
• If there are multiple valid approaches, compare them and explain the trade-offs.
• At the end, explicitly tell me what information you still need from me, if any.

The goal is not merely to describe the idea.

The goal is to produce a COMPLETE BLUEPRINT that a solo developer can use to build, secure, test, deploy, and launch the product in the real world.